Chapter 03: Control Flow

ENG3091

Khiem Nguyen

Contents

1 Control flow **2**
- 1.1 What is control flow? . . . 2
- 1.2 From Python to C++: a phrasebook . . . 2

2 `if` statements **3**
- 2.1 `if` and `if-else` . . . 3
- 2.2 Indentation means nothing in C++ . . . 3
- 2.3 Fix it with a block . . . 4
- 2.4 Always use braces: two cautionary tales . . . 5
- 2.5 Implicit blocks . . . 5
- 2.6 What the compiler sees . . . 5
- 2.7 `else if` chains . . . 6
- 2.8 `if`, `if` versus `if`, `else if` . . . 7
- 2.9 `=` is not `==` . . . 8

3 `switch` statements **9**
- 3.1 `switch`: testing one value against many . . . 9
- 3.2 `switch` syntax . . . 9
- 3.3 Fallthrough: the classic `switch` bug . . . 10
- 3.4 `break` stops the fall . . . 10
- 3.5 `break` in action . . . 11
- 3.6 `case` labels must be constants . . . 12
- 3.7 `goto`: just don't . . . 12

4 `while` loops **12**
- 4.1 `while` loops . . . 12
- 4.2 When the condition is false from the start . . . 13
- 4.3 Infinite loops . . . 14
- 4.4 Doing something every N iterations . . . 14
- 4.5 Nested loops . . . 15

5 `do-while` loops **16**
- 5.1 `do-while`: at least once . . . 16

6 `for` loops **16**
- 6.1 `for` loops . . . 16
- 6.2 A `for` loop is a tidy `while` loop . . . 17
- 6.3 Leaving parts out . . . 17
- 6.4 More than one counter . . . 18
- 6.5 Nested `for` loops . . . 19

7 `break` and `continue` 19
7.1 `break` and `continue` . . . 19
7.2 Breaking out of a loop . . . 20
7.3 Breaking out of an infinite loop . . . 20
7.4 `break` versus `return` . . . 21
7.5 `continue`: skip to the next iteration . . . 22
7.6 When `continue` makes code clearer . . . 22
7.7 When `continue` makes code clearer (continued) . . . 23

8 Questions for chocolate 23
8.1 Question 1: how does this program behave? . . . 23
8.2 Question 2: what's your opinion of this code? . . . 24
8.3 Question 2: my suggestion . . . 24

9 Wrapping up 25
9.1 Summary . . . 25
9.2 Your turn: two small challenges . . . 25
9.3 Keep learning . . . 26

1 Control flow

1.1 What is control flow?

You already know control flow from Python, so I won't explain what it is or what it's for. Here's what C++ offers:

Category	Implemented by
Conditionals	`if`, `else`, `switch`
Loops	`while`, `do-while`, `for`, range-based `for`
Jumps	`break`, `continue`, `goto`
Function calls	`function()`, `return`
Halts	`std::exit()`, `std::abort()`
Exceptions	`try`, `throw`, `catch`

Our goal: learn the ones you'll use every day, and point you to the rest. You don't need all of them to get an A1!

1.2 From Python to C++: a phrasebook

Python	C++
`if x > 0: … elif … else:`	`if (x > 0) { ... } else if ... else { ... }`
`True`, `False`	`true`, `false`
indentation marks a block	**braces** `{ }` mark a block
`x += 1`	`x += 1` or `++x`
`for i in range(10):`	`for (int i{0}; i < 10; ++i)`
`match`	`switch` (sort of)
(nothing)	`do { ... } while (...);`

The conditions go in **parentheses**, and the body goes in **braces**. The logical operators change too: `and`, `or` and `not` become `&&`, `||` and `!`.

2 `if` statements

2.1 `if` and `if-else`

```
if (condition) {
    true_statements;
}
```

```
if (condition) {
    true_statements;
} else {
    false_statements;
}
```

Listing 1 `example_if_else.cpp`

```
#include <iostream>

int main() {
    std::cout << "Enter a number: ";
    int x{};
    std::cin >> x;

    if (x > 10) {
        std::cout << x << " is greater than 10\n";
    } else {
        std::cout << x << " is not greater than 10\n";
    }
    return 0;
}
```

```
$ ./example_if_else
Enter a number: 12
12 is greater than 10
```

2.2 Indentation means nothing in C++

New programmers, and especially Python people, often write something like this:

Listing 2 `example_if_else_without_braces.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "Enter your height (in cm): ";
    int height{};
    std::cin >> height;

    if (height >= 140)
        std::cout << "You are tall enough to ride.\n";
    else
        std::cout << "You are not tall enough to ride.\n";
        std::cout << "Too bad!\n"; // in the else?

    return 0;
}
```

```
$ ./example_if_else_without_braces
Enter your height (in cm): 180
You are tall enough to ride.
Too bad!
```

Without braces, `if` and `else` each control **one** statement, so "Too bad!" always runs. g++ `-Wall` even warns: *this 'else' clause does not guard…*

2.3 Fix it with a block

Listing 3 `example_if_else_with_braces.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "Enter your height (in cm): ";
    int height{};
    std::cin >> height;

    if (height >= 140) {
        std::cout << "You are tall enough to ride.\n";
    } else { // the braces make both lines part of the else
        std::cout << "You are not tall enough to ride.\n";
        std::cout << "Too bad!\n";
    }
    return 0;
}
```

```
$ ./example_if_else_with_braces
Enter your height (in cm): 180
You are tall enough to ride.
```

Braces `{ }` turn several statements into one **block** (a *compound statement*), so both lines now belong to the `else`.

2.4 Always use braces: two cautionary tales

In a hurry, we add a line:

```
if (age >= min_drinking_age)
    purchase_beer();
    gamble(); // always runs!
```

Debugging, we comment out a line:

```
if (age >= min_drinking_age)
    // add_beer_to_cart();

check_out(); // now conditional!
```

The usual argument *against* braces is that they spread code out vertically, so you see less of it at once. My opinion: with today's big screens, that argument is obsolete.

> Best practice
>
> Put the statements of every `if` and `else` in braces, even single ones, at least while you're learning.

2.5 Implicit blocks

Without braces, the compiler puts the single statement in a block of its own anyway. So what does this print?

Listing 4 `define_variable_in_implicit_block_01.cpp`

```
#include <iostream>

int main() {
    if (true)
        int x{5};
    else
        int x{6};

    std::cout << x << '\n';
    return 0;
}
```

```
error: 'x' was not declared in this scope
```

Nothing: it doesn't compile! Each `x` lives in its own invisible block and is destroyed immediately, so no `x` is in scope on line 9. Remember **block scope** from Chapter 02?

2.6 What the compiler sees

The program on the last slide is exactly the same as this one:

Listing 5 `define_variable_in_implicit_block_02.cpp`

```cpp
#include <iostream>

int main() {
    if (true) {
        int x{5};
    } // this x is destroyed here
    else {
        int x{6};
    } // and this x is destroyed here

    std::cout << x << '\n'; // no x is in scope here
    return 0;
}
```

2.7 `else if` chains

Listing 6 `print_digit_name_ifelse.cpp`

```cpp
#include <iostream>

void print_digit_name(int x) {
    if (x == 1) {
        std::cout << "One\n";
    } else if (x == 2) {
        std::cout << "Two\n";
    } else if (x == 3) {
        std::cout << "Three\n";
    } else {
        std::cout << "Unknown\n";
    }
}

int main() {
    print_digit_name(2);
    return 0;
}
```

```
Two
```

C++ has no `elif`: write `else if` in full.

2.8 `if`, `if` versus `if`, `else if`

Listing 7 `if_if_vs_else_if.cpp`

```cpp
#include <iostream>

int main() {
    int score{85};

    std::cout << "if, if:\n";
    if (score >= 80) {
        std::cout << "  A\n";
    }
    if (score >= 60) { // checked as well, and also true
        std::cout << "  B\n";
    }

    std::cout << "if, else if:\n";
    if (score >= 80) {
        std::cout << "  A\n";
    } else if (score >= 60) { // skipped: a branch already ran
        std::cout << "  B\n";
    }
    return 0;
}
```

```
if, if:
  A
  B
if, else if:
  A
```

Separate `if`s check **every** condition; an `else if` chain stops at the **first** true one.

Which to use? It depends on the logic you want. Practice and you'll see!

2.9 = is not ==

Listing 8 `assignment_in_condition.cpp`

```cpp
#include <iostream>

int main() {
    int x{5};

    if (x = 0) { // oops: = assigns, == compares
        std::cout << "x is zero\n";
    } else {
        std::cout << "x is not zero\n";
    }
    std::cout << "x is now " << x << '\n';
    return 0;
}
```

```
x is not zero
x is now 0
```

- `x = 0` **assigns** 0 to `x`, and the condition is then the value `0`, which counts as `false`. Our variable is clobbered too!
- It compiles. Only `-Wall` saves you: *suggest parentheses around assignment used as truth value.*

3 `switch` statements

3.1 `switch`: testing one value against many

Listing 9 `print_digit_name_switch.cpp`

```cpp
#include <iostream>

void print_digit_name(int x) {
    switch (x) {
    case 1:
        std::cout << "One\n";
        return;
    case 2:
        std::cout << "Two\n";
        return;
    case 3:
        std::cout << "Three\n";
        return;
    default:
        std::cout << "Unknown\n";
        return;
    }
}

int main() {
    print_digit_name(2);
    return 0;
}
```

```
Two
```

Same job as the `else if` chain, but easier to read.

3.2 `switch` syntax

```cpp
switch (expression) {
case x:
    // statements
    break;
case y:
    // statements
    break;
default:
    // statements
}
// carries on here
```

1. The `expression` is evaluated.
2. If its value matches a `case` label, execution jumps to that label.
3. If nothing matches, execution jumps to `default` (if there is one).

4. If nothing matches and there's no `default`, the whole `switch` is skipped.

The expression must have an **integral** type, such as `int` or `char` (or an enumeration, later). `double` and strings are not allowed.

3.3 Fallthrough: the classic `switch` bug

Listing 10 `switch_fallthrough.cpp`

```cpp
#include <iostream>

int main() {
    int x{2};

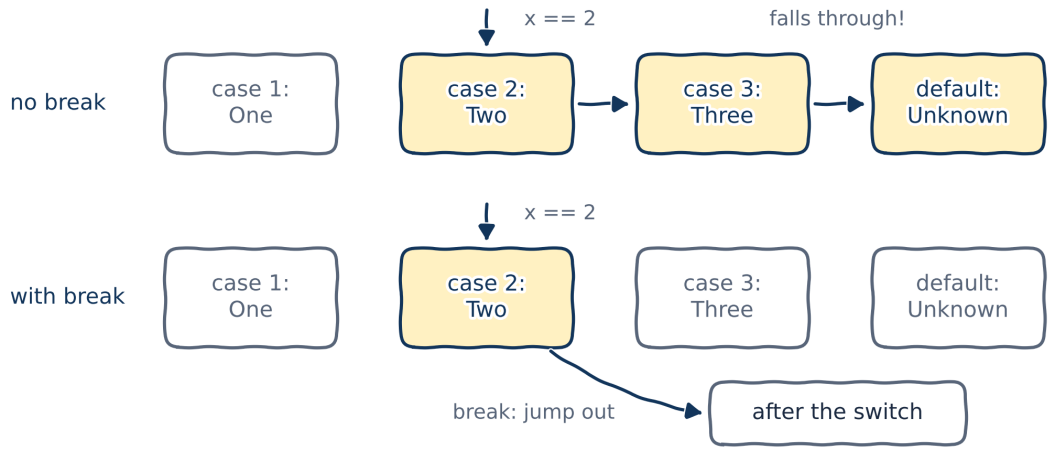
    switch (x) {
    case 1:
        std::cout << "One\n";
    case 2:
        std::cout << "Two\n"; // x matches here...
    case 3:
        std::cout << "Three\n"; // ...but this runs too
    default:
        std::cout << "Unknown\n"; // ...and so does this!
    }
    return 0;
}
```

```
Two
Three
Unknown
```

`-Wextra` warns: *this statement may fall through.*

A `case` label only marks where execution **starts**. Once it starts, it carries on through every case below, until something stops it.

3.4 `break` stops the fall



`break` means "done: jump to the end of the `switch`". Put one at the end of every case. (In our first example, `return` did the same job, because it left the whole function.)

3.5 `break` in action

Listing 11 `break_a_switch.cpp`

```cpp
#include <iostream>

void print_math(int x, int y, char op) {
    switch (op) {
    case '+':
        std::cout << x << " + " << y << " = " << x + y << '\n';
        break; // done: jump to the end of the switch
    case '-':
        std::cout << x << " - " << y << " = " << x - y << '\n';
        break;
    case '/':
        std::cout << x << " / " << y << " = " << x / y << '\n';
        break;
    }
}

int main() {
    print_math(7, 2, '+');
    print_math(7, 2, '-');
    print_math(7, 2, '/');
    return 0;
}
```

```
7 + 2 = 9
7 - 2 = 5
7 / 2 = 3
```

> **Coming from Python**
>
> `7 / 2` is `3` here, not `3.5`: dividing one `int` by another drops the fraction, like Python's `7 // 2`.

3.6 `case` labels must be constants

Listing 12 `case_label_expression.cpp`

```cpp
#include <iostream>

int main() {
    switch (1 - 2 + 3 - 4 + 5) { // any integer expression
    case 3:
        std::cout << "Three\n";
        break;
    case 2 + 2: // a constant expression works as a label
        std::cout << "Four\n";
        break;
    default:
        std::cout << "Something else\n";
        break;
    }
    return 0;
}
```

```
Three
```

- The `switch` expression can be any integer expression: here, `1 - 2 + 3 - 4 + 5` is 3.
- Each `case` label must be a **constant expression**: `2 + 2` is fine, a variable is not.

3.7 `goto`: just don't

- C++ has a `goto` statement that jumps to a labelled line anywhere in a function.
- It makes code very hard to follow, and loops plus `break` and `continue` almost always do the job better.
- Avoid it unless you have a truly compelling reason. If you're curious: `goto` statements on learncpp.com.

4 `while` loops

4.1 `while` loops

```cpp
while (condition) {
    statements;
}
```

1. Evaluate `condition`.
2. If it's `true`, run the body, then go back to step 1.
3. If it's `false`, carry on after the loop.

Listing 13 `while_count.cpp`

```cpp
#include <iostream>

int main() {
    int count{1};
    while (count <= 10) {
        std::cout << count << ' ';
        ++count; // add 1 to count
    }
    std::cout << "done!\n";
    return 0;
}
```

```
1 2 3 4 5 6 7 8 9 10 done!
```

`++count` adds 1 to `count`, like `count += 1`.

4.2 When the condition is false from the start

Listing 14 `while_loop_ex.cpp`

```cpp
#include <iostream>

int main() {
    int count{15};
    while (count <= 10) { // false from the start
        std::cout << count << ' ';
        ++count;
    }
    std::cout << "done!\n";
    return 0;
}
```

```
done!
```

- If the condition is `false` the first time, the body never runs at all.
- Sometimes that's a bug; often it's exactly what we want. "Process every item in the list" should do nothing at all for an empty list.

4.3 Infinite loops

Listing 15 `infinite_loop.cpp`

```cpp
#include <iostream>

int main() {
    while (true) { // loops forever...
        std::cout << "Loop again (y/n)? ";
        char c{};
        std::cin >> c;
        if (c == 'n') {
            return 0; // ...until this ends main()
        }
    }
}
```

```
$ ./infinite_loop
Loop again (y/n)? y
Loop again (y/n)? y
Loop again (y/n)? n
```

- A condition that is always `true` makes an **infinite loop**. Write them on purpose, with a way out, never by accident!
- A runaway program stuck in a loop? Press **Ctrl+C** in the terminal to stop it.

4.4 Doing something every *N* iterations

Listing 16 `program_using_if_and_while.cpp`

```cpp
#include <iostream>

int main() {
    int count{1};
    while (count <= 50) {
        if (count < 10) {
            std::cout << '0'; // pad single digits: 01, 02, ...
        }
        std::cout << count << ' ';

        if (count % 10 == 0) { // every 10th number...
            std::cout << '\n'; // ...start a new line
        }
        ++count;
    }
    return 0;
}
```

```
01 02 03 04 05 06 07 08 09 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
```

```
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
```

`%` gives the remainder after division, as in Python: `count % 10 == 0` is true for 10, 20, 30, …

4.5 Nested loops

Listing 17 `nested_loop_example.cpp`

```cpp
#include <iostream>

int main() {
    int outer{1};
    while (outer <= 5) {
        int inner{1}; // created afresh for every row
        while (inner <= outer) {
            std::cout << inner << ' ';
            ++inner;
        }
        std::cout << '\n';
        ++outer;
    } // inner is destroyed at the end of each row
    return 0;
}
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

For each turn of the outer loop, the inner loop runs to completion.

5 `do-while` loops

5.1 `do-while`: at least once

Listing 18 `do_while_loop_example.cpp`

```cpp
#include <iostream>

int main() {
    int selection{}; // outside the loop: used after it

    do {
        std::cout << "Please make a selection:\n";
        std::cout << "1) Addition\n";
        std::cout << "2) Subtraction\n";
        std::cin >> selection;
    } while (selection < 1 || selection > 2);

    std::cout << "You selected option " << selection << '\n';
    return 0;
}
```

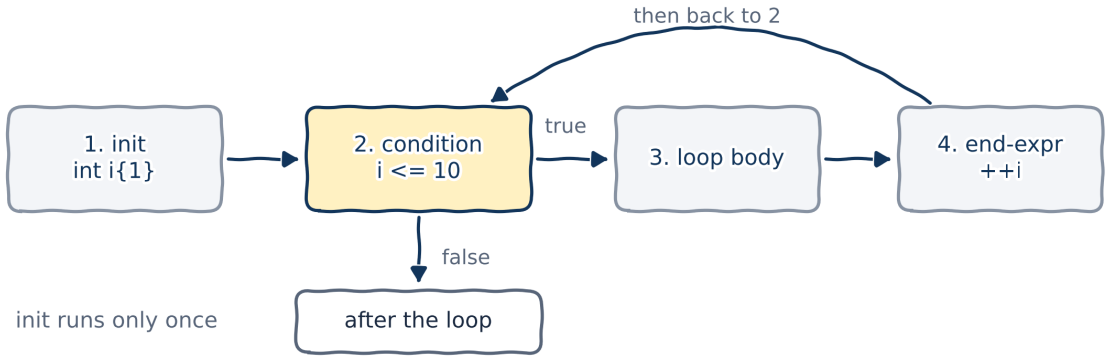
```
$ ./do_while_loop_example
Please make a selection:
1) Addition
2) Subtraction
7
Please make a selection:
1) Addition
2) Subtraction
2
You selected option 2
```

- The condition is checked at the **end**, so the body always runs at least once. Perfect for "ask until the answer is valid".
- Don't forget the semicolon after `while (...)`. Python has no `do-while`.

6 `for` loops

6.1 `for` loops

```cpp
for (init-statement; condition; end-expression) {
    statements;
}
```



The init-statement runs **once**. Then: condition, body, end-expression, condition, body, end-expression… until the condition is `false`.

6.2 A `for` loop is a tidy `while` loop

Listing 19 `for_loop_simple.cpp`

```cpp
#include <iostream>

int main() {
    for (int i{1}; i <= 10; ++i) {
        std::cout << i << ' ';
    }
    std::cout << '\n';
    return 0;
}
```

```cpp
{ // this block limits the scope of i
    int i{1};          // init-statement
    while (i <= 10) {  // condition
        std::cout << i << ' ';
        ++i;           // end-expression
    }
} // i is destroyed here
```

```
1 2 3 4 5 6 7 8 9 10
```

> **Watch out**
>
> `i <= 10` runs 10 times starting from 1; `i < 10` runs only 9. Getting this wrong by one is so common it has a name: an **off-by-one error**.

6.3 Leaving parts out

Every part of a `for` statement is optional:

```cpp
int i{0};
for (; i < 10;) { // no init, no end-expression
    std::cout << i << ' ';
```

```
    ++i;
}
```

```
for (;;) { // loops forever
    statements;
}
// same as while (true) { ... }
```

Legal, but rarely clearer. If you leave out the init and the end-expression, a `while` loop says the same thing more plainly.

6.4 More than one counter

Listing 20 `for_loop_multi_counters.cpp`

```
#include <iostream>

int main() {
    for (int x{0}, y{9}; x < 10; ++x, --y) {
        std::cout << x << ' ' << y << '\n';
    }
    return 0;
}
```

A comma separates the counters, both in the init-statement and in the end-expression.

```
0 9
1 8
2 7
3 6
4 5
5 4
6 3
7 2
8 1
9 0
```

6.5 Nested `for` loops

Listing 21 `nested_for_loops.cpp`

```cpp
#include <iostream>

int main() {
    for (char c{'a'}; c <= 'e'; ++c) { // outer loop: letters
        std::cout << c;                // print the letter first

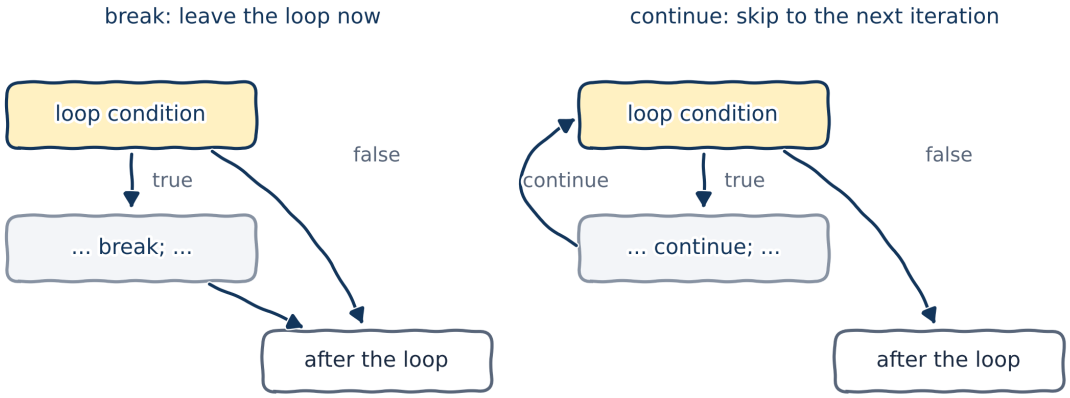
        for (int i{0}; i < 3; ++i) { // inner loop: numbers
            std::cout << i;
        }
        std::cout << '\n';
    }
    return 0;
}
```

```
a012
b012
c012
d012
e012
```

A `char` can be a loop counter too: `'a'`, `'b'`, `'c'`, …

7 `break` and `continue`

7.1 `break` and `continue`



- `break` ends the whole loop (or `switch`); execution continues after it.
- `continue` ends only the **current iteration** and jumps to the next one. In a `for` loop, the end-expression still runs.
- Both work just as they do in Python, and both usually sit inside an `if`.

7.2 Breaking out of a loop

Listing 22 break_a_loop.cpp

```cpp
#include <iostream>

int main() {
    int sum{0};

    for (int count{0}; count < 10; ++count) { // at most 10
        std::cout << "Number to add (0 to stop): ";
        int num{};
        std::cin >> num;

        if (num == 0) {
            break; // leave the loop right now
        }
        sum += num; // same as sum = sum + num
    }

    // execution continues here after the break
    std::cout << "The sum is " << sum << '\n';
    return 0;
}
```

```
$ ./break_a_loop
Number to add (0 to stop): 3
Number to add (0 to stop): 5
Number to add (0 to stop): 0
The sum is 8
```

7.3 Breaking out of an infinite loop

Listing 23 break_infinite_loop.cpp

```cpp
#include <iostream>

int main() {
    while (true) { // an intentional infinite loop
        std::cout << "Enter 0 to exit: ";
        int num{};
        std::cin >> num;

        if (num == 0) {
            break;
        }
    }
    std::cout << "We're out!\n";
    return 0;
}
```

```
$ ./break_infinite_loop
Enter 0 to exit: 5
Enter 0 to exit: 0
We're out!
```

A `while (true)` loop with a `break` is the usual way to write "keep going until something happens".

7.4 `break` versus `return`

Listing 24 `break_vs_return.cpp`

```cpp
#include <iostream>

int break_or_return() {
    while (true) {
        std::cout << "Type b (break) or r (return): ";
        char ch{};
        std::cin >> ch;
        if (ch == 'b') {
            break; // continue after the loop
        }
        if (ch == 'r') {
            return 1; // leave the whole function now
        }
    }
    std::cout << "We broke out of the loop\n";
    return 0;
}

int main() {
    int result{break_or_return()};
    std::cout << "It returned " << result << '\n';
    return 0;
}
```

```
$ ./break_vs_return
Type b (break) or r (return): b
We broke out of the loop
It returned 0
$ ./break_vs_return
Type b (break) or r (return): r
It returned 1
```

`break` leaves the **loop**; `return` leaves the whole **function**.

7.5 `continue`: skip to the next iteration

Listing 25 `continue_to_print_even.cpp`

```cpp
#include <iostream>

int main() {
    for (int count{0}; count < 10; ++count) {
        if (count % 2 == 1) { // odd number?
            continue;         // skip the rest of this iteration
        }
        std::cout << count << ' ';
    } // continue jumps to here, then ++count runs
    std::cout << '\n';
    return 0;
}
```

```
0 2 4 6 8
```

For an odd number, `continue` skips the printing. The end-expression `++count` still runs, so the loop moves on to the next number.

7.6 When `continue` makes code clearer

Some people say `break` and `continue` are just `goto` in disguise and should be avoided. Today's consensus: **use them when they make code easier to read.** Compare:

Listing 26 `nested_ifs.cpp`

```cpp
#include <iostream>

int main() {
    // Print the numbers below 50 that are even AND multiples of 3
    for (int n{1}; n < 50; ++n) {
        if (n % 2 == 0) {
            if (n % 3 == 0) {
                std::cout << n << ' '; // the real work, two levels deep
            }
        }
    }
    std::cout << '\n';
    return 0;
}
```

7.7 When `continue` makes code clearer (continued)

Listing 27 `guard_clauses.cpp`

```cpp
#include <iostream>

int main() {
    // Same job, but skip early with continue
    for (int n{1}; n < 50; ++n) {
        if (n % 2 != 0) {
            continue; // not even: skip
        }
        if (n % 3 != 0) {
            continue; // not a multiple of 3: skip
        }
        std::cout << n << ' '; // the real work, at the top level
    }
    std::cout << '\n';
    return 0;
}
```

```
6 12 18 24 30 36 42 48
```

Same output, but the checks sit up front and the real work stays at the top level instead of being buried in nested `if`s.

8 Questions for chocolate

8.1 Question 1: how does this program behave?

Listing 28 `Q_and_A_01.cpp`

```cpp
#include <iostream>

int main() {
    int count{0};
    while (count < 10) {
        if (count == 5) {
            continue;
        }
        std::cout << count << '\n';
        ++count;
    }
    return 0;
}
```

```
0
1
2
```

```
3
4
```

…and then nothing, ever. When `count` reaches 5, `continue` skips `++count`, so `count` stays 5 forever: an accidental infinite loop. Ctrl+C to the rescue!

8.2 Question 2: what's your opinion of this code?

Just say something: whatever you think!

Listing 29 `Q_and_A_02.cpp`

```cpp
#include <iostream>

int main() {
    int count{0};             // how many times we've looped
    bool keep_looping{true}; // controls whether the loop ends
    while (keep_looping) {
        std::cout << "Enter e to exit: ";
        char ch{};
        std::cin >> ch;

        if (ch == 'e') {
            keep_looping = false;
        } else {
            ++count;
            std::cout << "We've looped " << count << " times\n";
        }
    }
    return 0;
}
```

8.3 Question 2: my suggestion

It works, but the `keep_looping` variable only exists to end the loop. `break` says the same thing directly:

Listing 30 `Q_and_A_02_rewrite.cpp`

```cpp
#include <iostream>

int main() {
    int count{0}; // how many times we've looped
    while (true) {
        std::cout << "Enter e to exit: ";
        char ch{};
        std::cin >> ch;

        if (ch == 'e') {
            break;
        }
        ++count;
        std::cout << "We've looped " << count << " times\n";
    }
    return 0;
}
```

9 Wrapping up

9.1 Summary

- Conditions go in parentheses, bodies in braces. Indentation means nothing to the compiler, so **always use braces**.
- `else if`, not `elif`; `&&`, `||`, `!`, not `and`, `or`, `not`; and `==`, not `=`, when comparing.
- `switch` tests one integral value against constant `case` labels. End each case with `break`, or it falls through.
- `while` checks first; `do-while` runs at least once; `for` keeps init, condition and end-expression together.
- `break` leaves the loop, `continue` skips to the next iteration, `return` leaves the function.
- Compile with `-Wall -Wextra`: it caught misleading indentation, `=` in a condition, and fallthrough today.

9.2 Your turn: two small challenges

Go home, don't sleep yet, and write programs that print these. Try loops of each kind!

```
6 5 4 3 2 1
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

```
          1
        2 1
      3 2 1
    4 3 2 1
  5 4 3 2 1
6 5 4 3 2 1
```

9.3 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Fundamental data types in more depth: integers, floating point numbers, Booleans and chars
2. Logical operators (`&&`, `||`, `!`)
3. `switch` fallthrough and scoping

Next up, Chapter 04: constants and strings.